Meeting 14. Exploring Data with Matrix Products

A table can contain more structure than we can see by reading its rows. Movie ratings provide a useful example. We can ask which movies share an audience, which people have watched similar movies, or which movie features tend to occur together. Matrix products answer all three questions, provided we first decide what the rows, columns, and entries mean.

The arithmetic is often the easy part. A product of two matrices will happily compare missing ratings as though they were real zeros, or compare counts when we intended to compare preferences. A useful calculation therefore begins with a representation and ends with an interpretation.

Companions. The Meeting 14 notebook develops these computations on MovieLens. Explore Correlation, in pictures and Two products for interactive views. Code boxes and self-check answers expand when selected; they are open in the PDF.

The main points

1. **The entries determine what a comparison means**

   A rating matrix, a has-rated matrix, and a matrix of centered ratings answer different questions. A missing rating is not a zero-star rating. Section 1.

2. **Matrix products aggregate measurements**

   A vector of ones computes sums. An indicator matrix groups movies by genre. The denominator determines whether movies or individual ratings receive equal weight. Section 2.

3. **Correlation is a product of standardized columns**

   With population standard deviations, the correlation matrix is the average of the outer products of standardized rows. Constant columns require separate treatment. Section 3.

4. **The two Gram matrices compare different objects**

   A matrix times its transpose compares rows; its transpose times the matrix compares columns. Check both the dimensions and the meaning of one entry. Section 4.

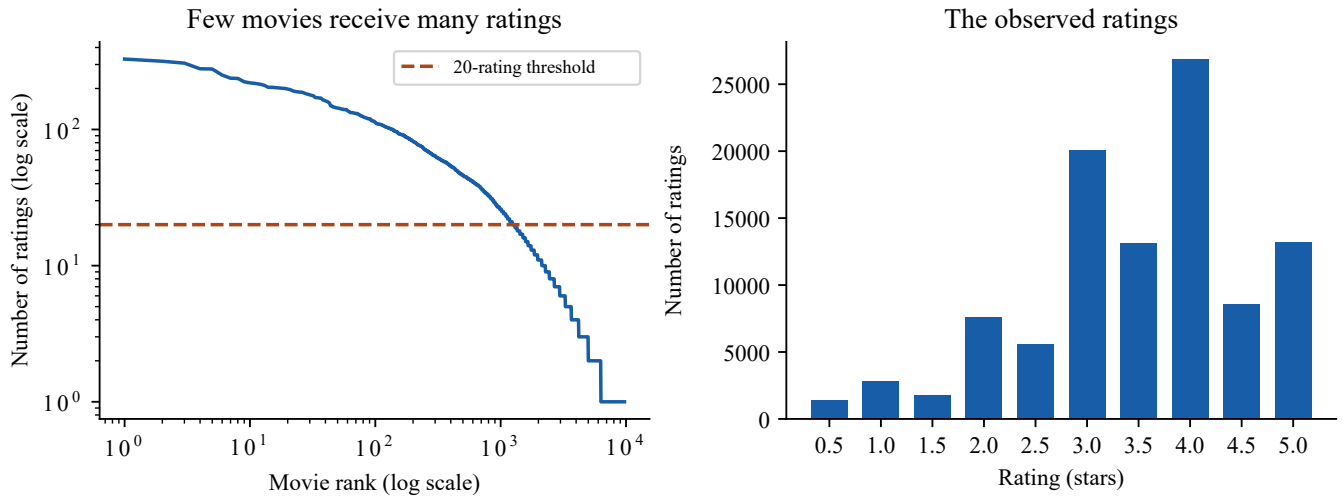
5. **A similarity score expresses a modeling choice**

   Normalizing lengths and centering ratings changes the neighbors of a movie. Report the representation, the score, and the evidence behind a claim. Sections 5-6.

1. From a list of ratings to a matrix

The course copy of MovieLens contains **100,836 ratings** from **610 people**. Its catalog lists 9,742 movies, of which 9,724 have at least one rating. Each observed rating has a person identifier, a movie identifier, a value from half a star to five stars, and a timestamp. A long table stores only the observed events; a rectangular rating matrix reserves a cell for every person-movie pair.

Let R_{ij} be the rating person i gave movie j. An unobserved entry is stored as `NaN`, a marker for missing data. We do not know whether the person disliked the movie, never watched it, or watched it without recording a rating. Assigning zero stars would invent information.



The actual course dataset. The left plot uses logarithmic axes. A threshold on the number of ratings selects a subset of movies; it does not make the sample representative of all viewers.

Following the notebook, retain movies with at least 20 ratings. This leaves **1,297 movie columns** and 610 person rows. There are 67,898 observed ratings among 791,170 possible cells: **8.58%** of the cells are observed. The retained movies account for 67.34% of all ratings. These are computed counts from the stored course data, not a claim about the current MovieLens service.

Beside R, construct a **has-rated matrix** B: an entry is 1 when a rating exists and 0 otherwise. Its zero means "no recorded rating." The entries of B describe observation, not approval.

Build the two matrices in pandas and NumPy

```
import numpy as np
import pandas as pd

base = "https://ikoutis.github.io/cs301-labs/unit-2/movies/"
ratings = pd.read_csv(base + "ratings.csv")
movies = pd.read_csv(base + "movies.csv")
count_by_movie = ratings.groupby("movieId").size()
kept_ids = count_by_movie[count_by_movie >= 20].index
kept = ratings[ratings.movieId.isin(kept_ids)]

table = kept.pivot(index="userId", columns="movieId", values="rating")
R = table.to_numpy()
B = (~np.isnan(R)).astype(float)
movie_ids = table.columns.to_numpy()
print(R.shape, B.mean())
```

The row and column labels establish the order of people and movies. Keep those labels with the array. Casting the indicator matrix to a numeric type ensures that matrix products count co-occurrences.

2. Features and aggregation

A column sum is a matrix product

A feature describes an object with a number. For a movie, examples include its number of ratings, mean rating, rating spread, release year, and genre indicators. The object changes when we change tables: rows of R are people, while rows of a movie-feature table are movies.

Let $\mathbf{1}$ be a column vector with one entry for each person, all equal to 1. Multiplying its transpose by B adds the rows of B. The result has one count for each movie:

$$c^{\mathsf{T}} = \mathbf{1}^{\mathsf{T}} B\,.$$

In words: The rating count vector equals a row vector of ones multiplied by the has-rated matrix. Its jth entry counts the people who rated movie j.

To compute movie means, replace missing ratings by zero *for the numerator only*, sum each column, and divide by its observed count. The denominator counts ratings, not all 610 people. Zero-filling a sum is harmless here because the mask supplies the correct denominator; zero-filling and taking an ordinary mean would be a different calculation.

Counts, means, and spread of each movie's ratings

```
count = B.sum(axis=0)
total = np.nan_to_num(R, nan=0.0).sum(axis=0)
mean = total / count
spread = np.nanstd(R, axis=0, ddof=0)

assert np.allclose(np.ones(R.shape[0]) @ B, count)
assert np.allclose(mean, np.nanmean(R, axis=0))
```

The threshold guarantees a positive count for every retained movie. A mean based on 20 ratings and a mean based on 300 ratings have different amounts of supporting evidence. Neither number alone measures how well the movie will suit a particular viewer.

An indicator matrix groups movies

Construct G with one row per retained movie and one column per genre. Set $G_{ja} = 1$ when movie j has genre a. A movie can have several genres, so a row can contain several ones. The course data gives 19 nonempty genre columns.

Multiplying G^{T} by the vector of movie means adds those means within each genre. Dividing by the number of movies in the genre gives a **mean of movie means**. Each movie receives equal weight in that calculation. If instead we add all individual rating values and divide by all rating counts within a genre, heavily rated movies receive more weight. These two questions can have different answers.

> **Read the denominator.**
>
> "Average rating of a genre" is incomplete without specifying what is being averaged. Equal weight per movie and equal weight per rating are both meaningful, but they describe different summaries. Genre overlap also means these groups are not disjoint.

3. Correlation from standardized columns

Center first, then put columns on a common scale

Suppose X is a complete numeric table with n observations and d features. The mean μ_a locates feature a; its standard deviation σ_a measures spread. Subtracting the mean centers the column. Dividing by a nonzero standard deviation expresses values in units of that column's spread.

$$Z_{ia} = \frac{X_{ia} - \mu_a}{\sigma_a}.$$

In words: A standardized entry is the original entry minus its feature mean, divided by that feature's standard deviation.

Here standard deviations use divisor n, the convention `ddof=0` in NumPy. Each standardized column has mean zero and average squared value one. A constant column has standard deviation zero: its correlation is undefined, so do not divide by zero and silently turn the result into a meaningful score.

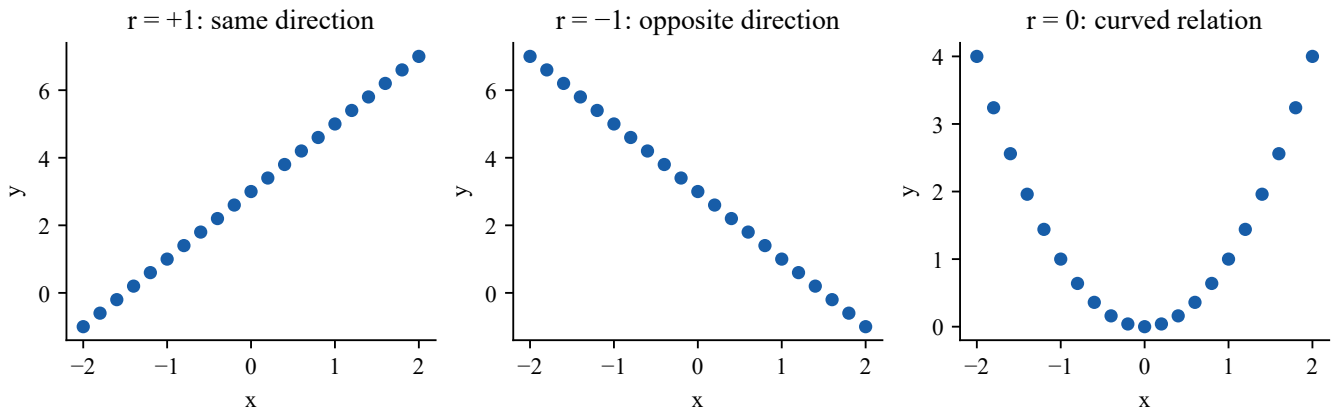
One entry is an average of products

For two standardized columns, multiply corresponding entries and average. Observations where both are above their means, or both below, contribute positively. Observations where one is above and the other below contribute negatively. The whole collection of these averages is one matrix product:

$$C = \frac{Z^{\top} Z}{n}, \qquad C_{ab} = \frac{1}{n} \sum_{i=1}^{n} Z_{ia} Z_{ib} \,.$$

In words: The correlation matrix equals the transpose of the standardized data matrix times that matrix, divided by the number of observations. Entry a, b is the average product of standardized features a and b.

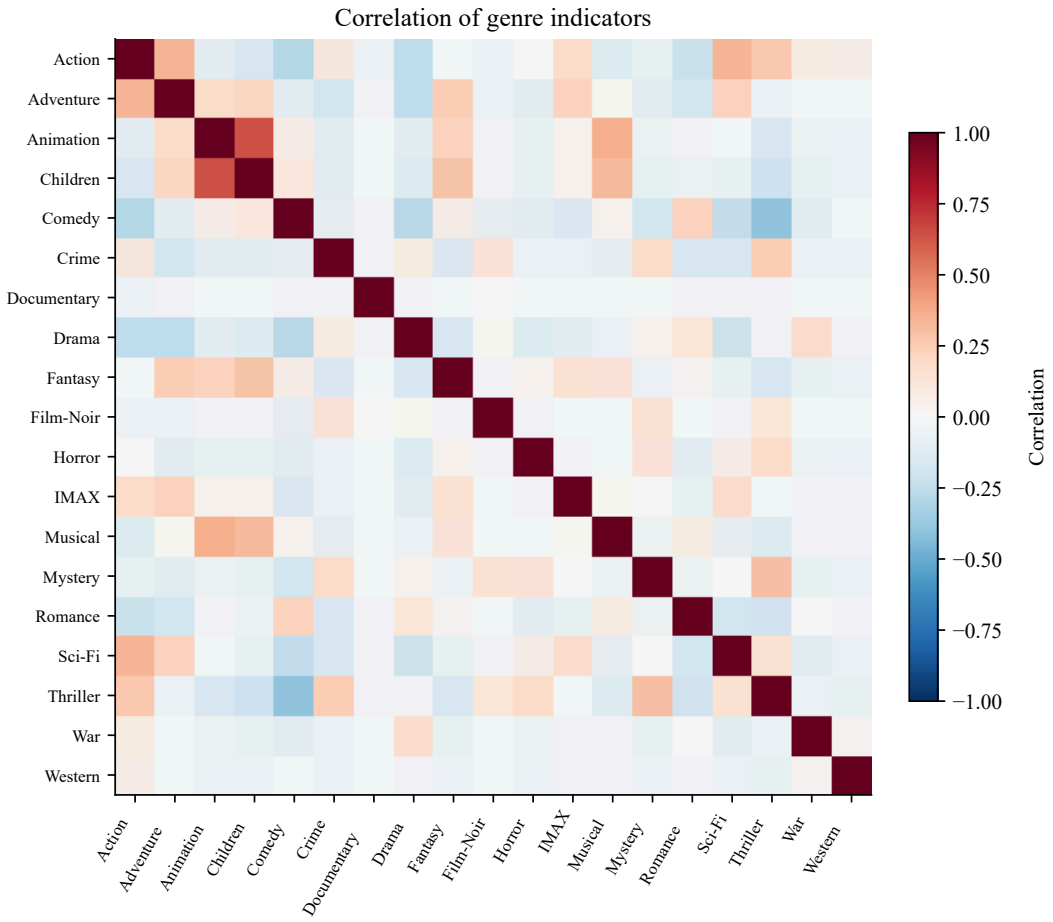
The result is $d \times d$, symmetric, with 1 on the diagonal for every nonconstant feature. Each off-diagonal entry lies between -1 and 1. If standard deviations instead use divisor $n - 1$, the matching correlation formula divides by $n - 1$. Mixing those conventions changes the diagonal.



Correlation measures linear association. A zero correlation does not rule out a strong nonlinear relationship; a nonzero correlation does not by itself establish causation.

Apply the product to genres

For the genre matrix, observations are movies and features are genre indicators. A positive correlation means that the two indicators tend to be above their means together, beyond what their individual frequencies would suggest. A negative correlation means their coexistence is less common relative to those frequencies. It does not mean the genres are opposites in every movie, or that one prevents the other.



Correlation of genre membership among the 1,297 retained movies. The calculation uses the same selected movies for every column; this is not a correlation of incomplete rating columns.

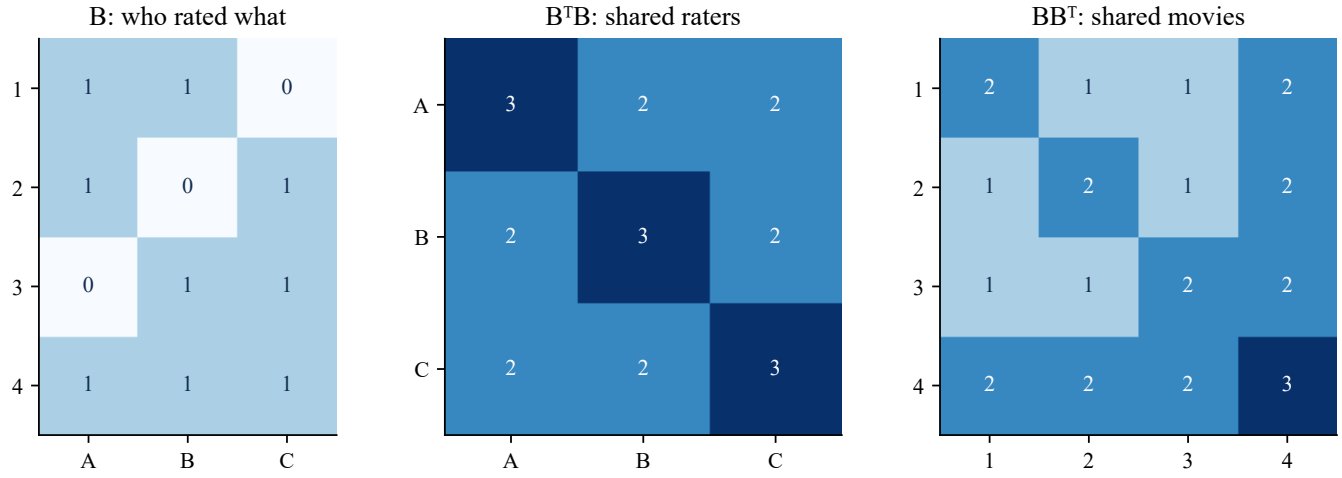
Compute and check the genre correlation matrix

```
G = (movies.set_index("movieId")
     .loc[movie_ids, "genres"].str.get_dummies(sep="|"))
G = G.loc[:, G.sum() > 0].astype(float)
sd = G.std(axis=0, ddof=0)
G = G.loc[:, sd > 0]  # correlation needs nonconstant columns
Z = (G - G.mean(axis=0)) / G.std(axis=0, ddof=0)
C = Z.to_numpy().T @ Z.to_numpy() / len(Z)
assert np.allclose(C, np.corrcoef(G.to_numpy(), rowvar=False))
```

The complete feature table matters. Applying this expression directly to R propagates missing values. Restricting each pair of rating columns to its common raters is another possible analysis, but then different entries may use different sets of people and have different support.

4. Two products, two questions

For $X \in \mathbb{R}^{n \times d}$, the product XX^T is $n \times n$. Entry (i, j) is the dot product of row i and row j. The product $X^\mathsf{T}X$ is $d \times d$, and entry (a, b) compares columns a and b. These are called **Gram matrices**.



A complete hand-checkable example: four people and three movies. A 1 records a rating, regardless of how many stars were given. Labels 1-4 are illustrative person numbers, not MovieLens identities.

> **Read one entry before interpreting the whole matrix**
>
> Columns A and B in the example are `[1, 1, 0, 1]` and `[1, 0, 1, 1]`. Their dot product is `1×1 + 1×0 + 0×1 + 1×1 = 2`: two people rated both movies. The A diagonal entry is 3 because three people rated A.
>
> Rows 1 and 2 are `[1, 1, 0]` and `[1, 0, 1]`. Their dot product is 1: these people share one rated movie. A diagonal row entry counts the movies rated by that person.

For the actual has-rated matrix, $B^\mathsf{T}B$ compares 1,297 movies using 610 person coordinates. BB^T compares 610 people using 1,297 movie coordinates. The diagonal entries are counts; off-diagonal entries are overlap counts. A large overlap need not mean similar taste: popular movies and active viewers tend to overlap with many others.

Storage can decide which product is feasible. A dense Gram matrix for n observations has n^2 entries even if the original data has relatively few features. Symmetry can save storage in a specialized representation, but an ordinary NumPy product returns the full dense array.

5. Normalize lengths; then ask what was normalized

Cosine similarity

A dot product is large when vectors point in similar directions, but also when their lengths are large. Cosine similarity divides out the lengths:

$$\text{cosine}\,(x, y) = \frac{x^{\mathsf{T}} y}{\|x\|\,\|y\|}.$$

In words: Cosine similarity is the dot product of two nonzero vectors divided by the product of their lengths.

For the toy has-rated columns A and B, the dot product is 2 and each length is $\sqrt{3}$, so cosine similarity is $2 / 3$. For indicator columns, this is the number of shared raters divided by the square root of the product of the two rating counts. It reduces the influence of popularity; it does not remove every sampling effect.

To compute all column cosines, divide each nonzero column of a matrix M by its length, producing U, and compute $U^{\mathsf{T}}U$. A zero-length column has no direction, so its cosine similarity is undefined and must be handled explicitly.

Three definitions of a similar movie

Representation	What an entry records	What its cosine emphasizes
Has-rated indicators	1 for an observed rating; 0 for missing	Overlap of the recorded audiences
Ratings, missing set to zero	Observed star value, otherwise 0	Shared raters and the sizes of their ratings
User-centered ratings	Observed rating minus that person's mean; missing set to 0	Whether shared viewers rate the movies above or below their own usual level

Centering by person addresses a particular difference: some people use high ratings frequently, while others use low ratings. In these notes, a person's mean uses that person's observed ratings among the retained movies, exactly as in the notebook. After subtraction, zero means no deviation from that baseline. Setting missing entries to zero gives them no positive or negative contribution; it still does not reveal an unobserved preference.

A centered column cosine is not automatically the Pearson correlation of two movie-rating columns. Here we center *rows* by person and zero-fill missing entries, then normalize complete columns. Pairwise Pearson correlation would center the two selected columns over their jointly observed ratings. The choices of baseline, population, and missing-data treatment differ.

Compute all three cosine matrices

```
def cosine_of_columns(M):
    lengths = np.linalg.norm(M, axis=0, keepdims=True)
    if np.any(lengths == 0):
        raise ValueError("A zero column has no cosine direction")
    U = M / lengths
    return U.T @ U

person_mean = np.nanmean(R, axis=1, keepdims=True)
centered = np.nan_to_num(R - person_mean, nan=0.0)
S_rated = cosine_of_columns(B)
S_given = cosine_of_columns(np.nan_to_num(R, nan=0.0))
S_centered = cosine_of_columns(centered)
```

Subtracting an array with shape `(610, 1)` broadcasts one mean across each person's row. Missing entries remain missing during subtraction and are filled afterward.

The following are the five nearest movies to *Toy Story (1995)* among the retained movies, excluding Toy Story itself. Each list is recomputed from the course data. The numbers are cosine similarities, rounded to three decimals.

Representation	Nearest movies and scores
Has-rated indicators	Independence Day (a.k.a. ID4) (1996) (0.585), Jurassic Park (1993) (0.584), Forrest Gump (1994) (0.579), Star Wars: Episode IV - A New Hope (1977) (0.577), Mission: Impossible (1996) (0.568)
Ratings with missing set to zero	Toy Story 2 (1999) (0.573), Jurassic Park (1993) (0.566), Independence Day (a.k.a. ID4) (1996) (0.564), Star Wars: Episode IV - A New Hope (1977) (0.557), Forrest Gump (1994) (0.547)
User-centered ratings	Toy Story 2 (1999) (0.351), Aladdin (1992) (0.282), Toy Story 3 (2010) (0.279), Wallace & Gromit: The Wrong Trousers (1993) (0.264), Incredibles, The (2004) (0.224)

Changing neighbors is evidence that "similar" depends on the representation. It is not evidence that one list must be correct for every viewer. A movie with very few shared raters can also receive an unstable score; examine overlap counts together with similarity.

6. From an output to a supported claim

A useful analysis states a claim narrowly enough that the computation can support it. "In this selected sample, these two genre indicators have positive correlation" names the sample and statistic. "One genre causes the other" adds a causal interpretation the calculation did not establish. Similarly, a nearest-neighbor list can suggest a recommendation, but it is not a measured prediction of a particular person's enjoyment.

Start with the objects and representation. Identify the statistic, show the relevant values or a labeled plot, and state one limitation. Check whether a conclusion depends on a few observations, on the 20-rating threshold, or on a centering choice. A change of ordering can reveal blocks in a heatmap, but reordering rows and columns does not change the pairwise numbers.

Self-checks and worked answers

1. A table has 100 people as rows and 8 features as columns. Which product compares people? XX^{T}, with shape 100×100. Its entries are row dot products. The other product is 8×8 and compares features.

2. Does a large entry in a has-rated Gram matrix show that two movies received similar ratings? No. It counts people who rated both. Their star values never entered the calculation.

3. Can a feature have correlation zero with another feature while determining it exactly? Yes. A symmetric collection of positive and negative values of x has zero correlation with x^2. The relationship is nonlinear.

4. Why does the user-centered matrix fill missing values after subtraction? The mean must be computed from observed ratings. Subtracting a mean from an invented zero-star rating would create a negative opinion where none was observed.

5. Why is a movie's self-similarity removed in a neighbor search? A nonzero column has cosine similarity 1 with itself. The task is to find other movies.

Data and further reading

The calculations use the course copy of the MovieLens small dataset, distributed by the University of Minnesota GroupLens group. See the dataset description and license and Harper and Konstan, The MovieLens Datasets: History and Context (2015). All numeric examples labeled “toy” or “illustrative” are constructed for explanation.